

In-Class Hands-On: Heap Exercise – with Solutions

This is a **group assignment** that you work on in groups of 2–3 persons. Solve each exercise and discuss the solution approach used as a group.

Write the solutions on a **piece of paper**. Please *close all computers and phones and put them aside*. Instead, please **call the teaching assistants (TAs) over if you need help**. The goal is to learn (it's not about all answers necessarily being correct; any reasonably thoughtful submission gets a full point).

Rewrite up your final answers clearly on a single sheet of paper, if needed (use both sides if necessary; you can get a second piece of paper). **Write the names of all group members on the top of the sheet(s).**

Hand in your answer sheet(s) to the TA.

You start with an empty *max-heap* (a complete binary tree where every parent is greater than or equal to its children), represented as an array.

1. Insert

Insert the following keys *in this order* into the max-heap: **15, 10, 20, 8, 30, 5, 25, 40, 12, 18**

- Draw the resulting heap as both a **tree diagram** and an **array** (index of root is at 0; *left-child* is at $2*parent_idx+1$; and right-child at $2*parent_idx+2$).
- In your own words, describe the approach used for inserting (be concise).

2. Remove-max

Using the heap from Question 1:

- a) What is the maximum element? Where is it stored, and why?
 - b) Perform *two* consecutive *remove-max* operations. Draw the heap after each extraction.
- In your own words, describe the approach used for remove the max (be as concise as possible). What “heap property restoration” step is needed, and why?
 - Write the “heap property restoration” up in pseudo-code as a function called *heapify_down(index)* where you work directly on the array representing the tree.

3. Heapify (Build-Heap)

Here you get to *design your own algorithm*! Given an *unsorted array*, say:

[3, 9, 2, 1, 7, 8, 4, 6, 5, 10],

build a max-heap in-place (i.e. just by swapping elements in the array). Write the algorithm up in pseudo-code, assuming you have access to the *heapify_down(index)* function from above that makes the element at location *index* “sink-down” to its rightful place in the heap.

4. Reflection

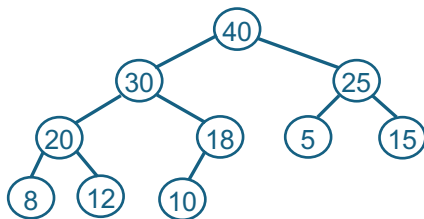
Answer the following short questions:

- a) A max-heap supports $O(1)$ find-max. Does it support $O(1)$ find-min? Why or why not?
- b) Is the array **[50, 30, 40, 20, 25, 35, 10]** a valid max-heap? Give your reasoning.

Question 1:

15, 10, 20, 8, 30, 5, 25, 40, 12, 18

[15]
[15, 10]
[15, 10, 20] => [20, 10, 15]
[20, 10, 15, 8]
[20, 10, 15, 8, 30] => [20, 30, 15, 8, 10] => [30, 20, 15, 8, 10]
[30, 20, 15, 8, 10, 5]
[30, 20, 15, 8, 10, 5, 25] => [30, 20, 25, 8, 10, 5, 15]
[30, 20, 25, 8, 10, 5, 15, 40] => [30, 20, 25, 40, 10, 5, 15, 8] => ... => [40, 30, 25, 20, 10, 5, 15, 8]
[40, 30, 25, 20, 10, 5, 15, 8, 12] => [40, 30, 25, 20, 10, 5, 15, 8, 12]
[40, 30, 25, 20, 10, 5, 15, 8, 12, 18] => [40, 30, 25, 20, 18, 5, 15, 8, 12, 10]
[40, 30, 25, 20, 18, 5, 15, 8, 12, 10]



Describe:

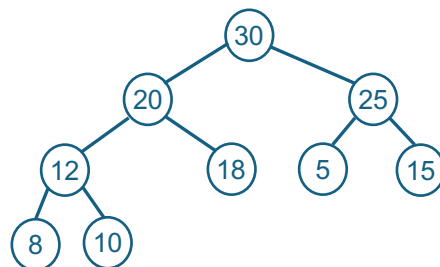
Add the new element as a new last node, then *heapify-up*, that is, repeatedly compare it with its parent; if it's larger, swap them and move up. Repeat until the element is smaller than its parent (or reaches the root).

Question 2:

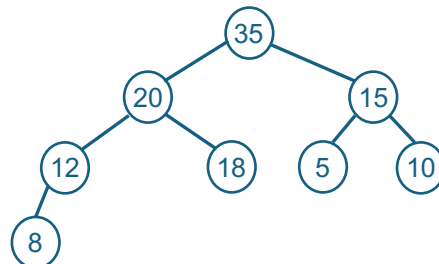
a) 40 is the maximum element, always stored at the root.

b)

[40, 30, 25, 20, 18, 5, 15, 8, 12, 10]
[10, 30, 25, 20, 18, 5, 15, 8, 12]
[30, 10, 25, 20, 18, 5, 15, 8, 12]
[30, 20, 25, 10, 18, 5, 15, 8, 12]
[30, 20, 25, 12, 18, 5, 15, 8, 10]



[10, 20, 25, 12, 18, 5, 15, 8]
[25, 20, 10, 12, 18, 5, 15, 8]
[25, 20, 15, 12, 18, 5, 10, 8]



Describe:

Element of last-node, put in root, last-node, removed, then *heapify-down* (*heap restoration property*). That is, repeatedly compare a value to that of its children; if either (or both) child larger, then swap (with the larger of the two children, if both are larger); repeat until no swapping needed (or at a leaf).

```

function heapify_down(index):
    n = length of heap array
    while True:
        left = 2 * index + 1
        right = 2 * index + 2
        largest = index
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest == index:
            break // heap property satisfied, stop
        swap(heap, index, largest)
        index = largest

```

Question 3:

```

function build_heap(array):
    n = length of array
    // Only need to look at internal nodes, thus  $n//2 - 1$ , go up to root
    for idx from (n // 2 - 1) down to 0:
        heapify_down(idx)

```

Question 4:

- a) No. In a max heap, min node is one of the leaves, would need to search amongst them.
- b) Yes, in the corresponding tree, child values are less than their parent.